

INF01151 – Sistemas Operacionais II

Aula 01 - Introdução à disciplina

Prof. Alberto Egon Schaeffer Filho

Objetivos da disciplina

- Estudo de problemas clássicos de programação concorrente
 - Contém duas ou mais tarefas executando "*ao mesmo tempo*"
 - Aspectos importantes: **sincronização** e **comunicação**
 - Transportados para várias situações reais
- Apresentação de problemas comuns a várias áreas:
 - Sistemas operacionais
 - Sistemas distribuídos
 - Sistemas de tempo real

Histórico: programação concorrente

1962

- Controladores de dispositivos de hardware
- Possibilitam que CPU execute nova aplicação
 - Enquanto operações de I/O são executadas para programas suspensos
- Comunicação entre controladores e processador através de interrupções

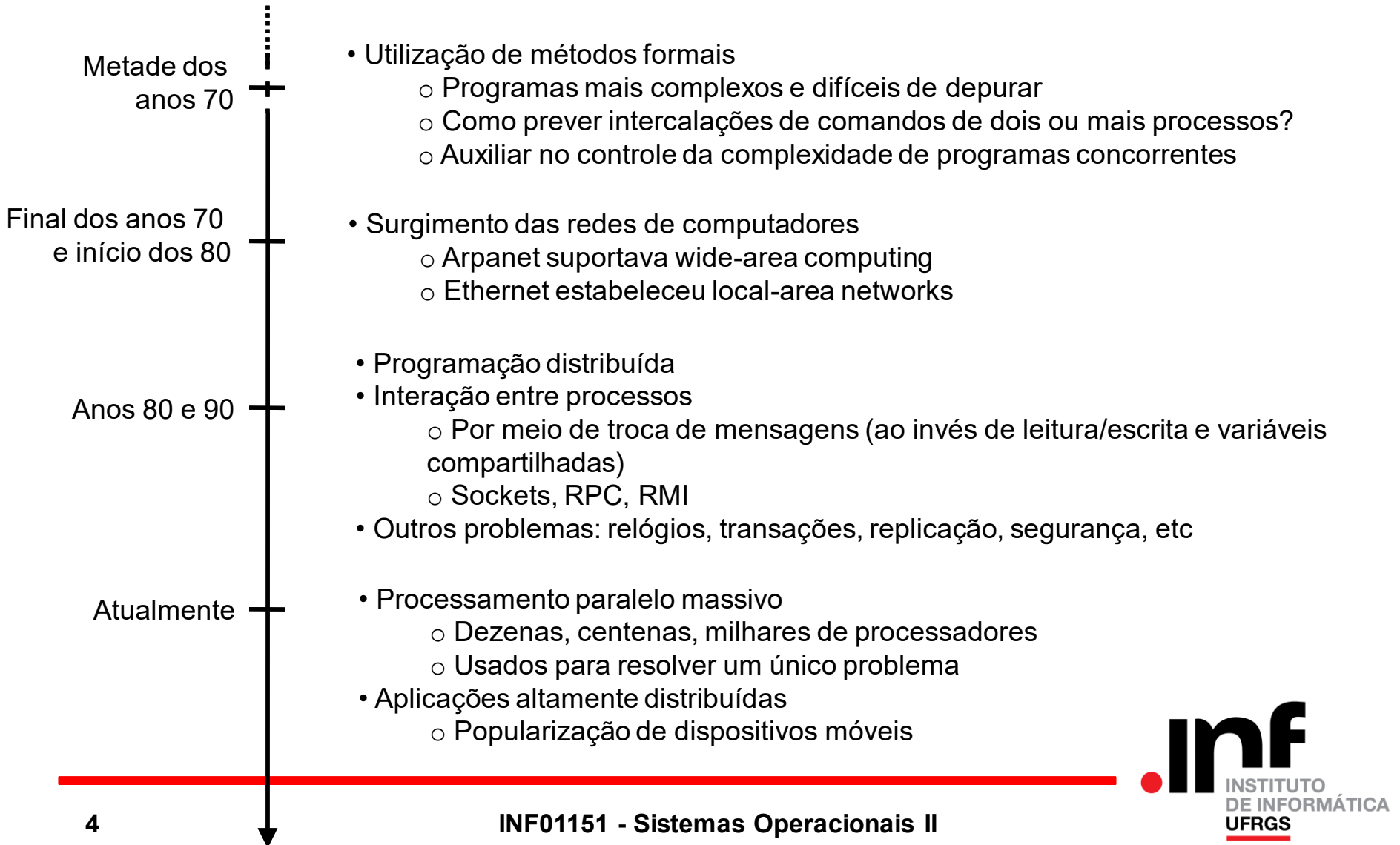
Concorrente = acontecendo ao mesmo tempo
Preocupação de projetistas de SO

- Primeiro grande desafio
- Problema da seção crítica
 - Se uma parte de um programa estiver atualizando uma variável, uma interrupção pode acontecer e outro processo pode tentar atualizar a mesma variável
- Desenvolvimento de primitivas de sincronização
 - Mutex, semáforos, monitores

Final dos
anos 60

- Surgimento de computadores com múltiplos processadores
 - Oportunidade para programadores de aplicações
 - Novo desafio para projetistas de SOs
- Diferentes programas executando ao mesmo tempo em diferentes processadores, ou
- Única aplicação executando mais rápido se re-escrita pra usar múltiplos processadores
 - Sincronização torna-se ainda mais difícil de gerenciar

Histórico: programação concorrente



Objetivos da disciplina (versão 2)

- Visão pragmática
 - Compreensão dos **conceitos fundamentais, mecanismos e construções** empregados para programação concorrente (do ponto de vista de sistemas operacionais)
 - Desenvolver **habilidade para solucionar problemas** de sincronização em programação concorrente
 - Compreender os **principais problemas** envolvidos no desenvolvimento de sistemas distribuídos
 - Conhecimento prático e uso das primitivas e **técnicas de programação** das atuais **linguagens de programação** concorrente

Súmula

- Conteúdo programático
 - **1ª Parte: Programação concorrente**
 - Introdução, especificação de concorrência, atomicidade e sincronização
 - Processos e threads, exclusão mútua, semáforos e monitores
 - Mecanismos de IPC (Inter Process Communication):
 - Tipos de comunicação (assíncrona/síncrona, RPC, RMI, web services)
 - **2ª Parte: Sistemas distribuídos**
 - Objetivos, conceitos de software e questões de projeto
 - Sincronização temporal, relógios físicos e relógios lógicos
 - Algoritmos distribuídos: eleição, exclusão mútua, deadlock
 - Comunicação em grupo e multicast
 - Outros tópicos: transações, replicação, segurança

Bibliografia

- Bibliografia base/essencial:
 - **Andrews, G.** – *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - **Silberschatz, A. and Galvin, P.** – *“Operating Systems Concepts”*. Addison-Wesley, 1994.
 - **Coulouris, G.; Dollimore, J.; Kindberg, T.** – *“Distributed Systems: Concepts and Design”* (5th edition), Addison-Wesley, 2012.
- Bibliografia complementar:
 - **Toscani, S. S.; Oliveira, R. S.; Carissimi, A.** – *“Sistemas Operacionais e Programação Concorrente”*. Sagra Luzzatto.

Introdução (light)

- Aspectos da computação concorrente
 - Multi-thread
 - Paralela
 - Distribuída
- Conhecimentos básicos
 - Conceitos de programação concorrente (INF01142)
 - Threads/Processos
 - Estados, execução, escalonamento
 - Problemas de sincronização
- Linguagens de programação
 - C & Java

Concorrência versus paralelismo

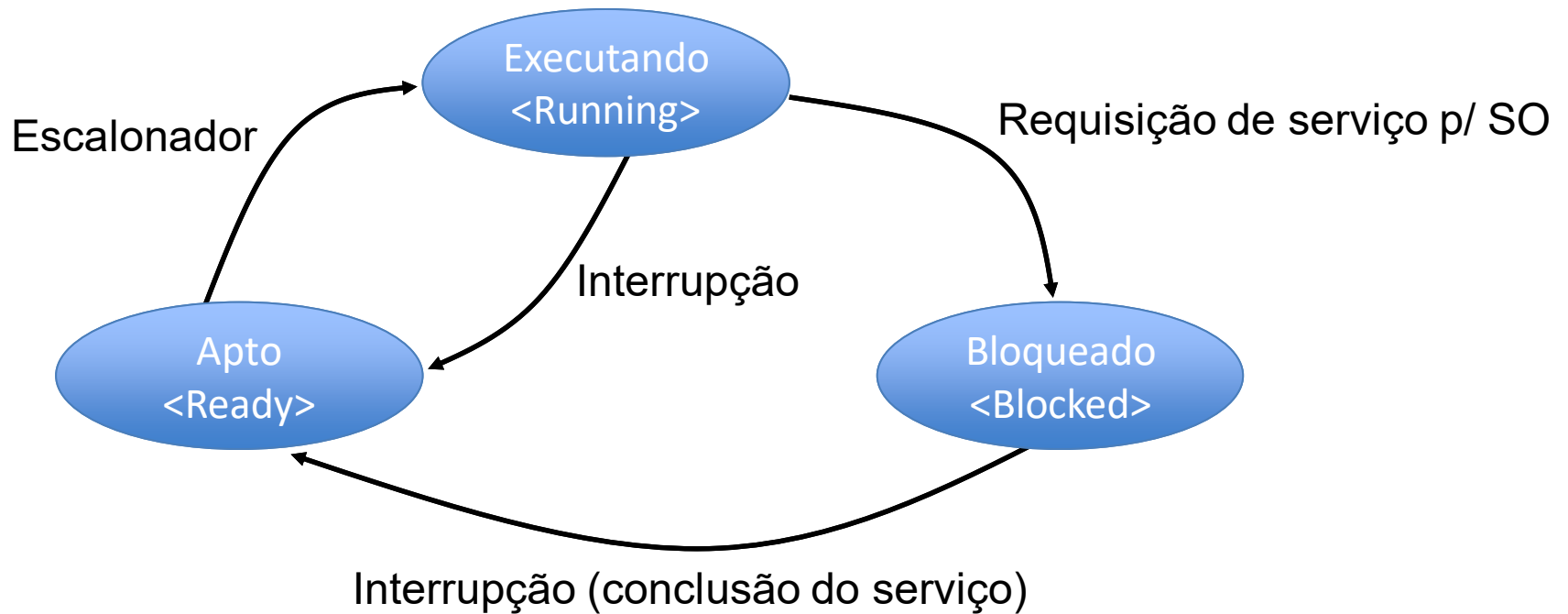
Concorrência $\neq$ Paralelismo

- Paralelo
 - Execução realizada fisicamente ao mesmo tempo
 - Com o objetivo de reduzir o tempo de resposta
 - Ex. simultaneamente em processadores paralelos
- Concorrente
 - Execução realizada logicamente ao mesmo tempo
 - Pode ser implementada sem nenhum paralelismo real

Multi-thread x paralela x distribuída

- Aplicação multi-thread
 - **O que é?** Possui mais de 1 thread (normalmente compartilhando CPU)
 - **Por que?** Melhor aproveitamento da CPU, boa forma de organizar sistemas de software modernos
- Aplicação paralela
 - **O que é?** Processos executando fisicamente ao mesmo tempo em diferentes processadores
 - **Por que?** Resolver um problema mais rapidamente -- ou resolver um problema maior
- Aplicação distribuída
 - **O que é?** Processos em diferentes *hosts* se comunicam através da rede
 - **Por que?** Distribuir trabalho, acessar dados inerentemente distribuídos, computação paralela em larga-escala sobre a rede

Estados de um processo



Escalonamento de processos

- Em arquiteturas Von Neumann os processos “parecem” executar ao mesmo tempo
 - Mas, na verdade, são intercalados (vale para threads também)
 - Fila de processos *ready* não é necessariamente FIFO...
- Escalonamento
 - Emprega conceitos de processo, chaveamento de contexto e política de escalonamento
 - *Dispatcher* passa o controle da CPU ao processo selecionado
 - Existem várias políticas
 - First-come, first-served
 - Shortest job first
 - Round robin
 - Priority

Escalonamento de processos

- Dois casos extremos
 - Não-preemptivo (cooperativo)
 - Um processo/thread executa até liberar o processador voluntariamente, ou terminar
 - Preemptivo
 - Um processo/thread é interrompido para permitir que outros processos/threads executem
- A grande maioria dos sistemas é preemptiva
 - Windows em versões recentes (Windows 3.x era não-preemptivo)
 - Mac OS X

Tipos de processos

- ~~Independentes~~
 - Não interessantes
- Competitivos
 - Típicos em sistemas operacionais modernos
 - Competem por recursos compartilhados da máquina
- Colaborativos
 - Processos executam para resolver um problema comum

Problemas comuns

- A competição leva:
 - Deadlock (impasses)
 - Starvation (inanição)
- Atomicidade
 - Uma ação dita atômica é aquela que tem a garantia de ser executada sem interrupção (do ponto de vista de outros processos)
 - A execução de processos/threads são intercaladas, como definir então ações atômicas?
 - Instruções individuais??
 - Blocos de instruções/código??

Exemplo de aplicação

- Produtor/consumidor
 - Frequentemente encontrado em programação concorrente
 - Principais componentes
 - Um processo produtor que produz itens
 - Um processo consumidor que consome itens
 - Um recurso compartilhado: objeto, buffer, banco de dados, etc
 - Principais problemas?
 - Itens produzidos podem ficar sem serem consumidos
 - Consumidor pode ler dados apenas parcialmente produzidos
 -
- Código exemplo em Java...

Exemplo de aplicação

Not looking so good...

```
Albertos-MacBook-Pro:ProducerConsumer alberto$ java ProducerConsumer
Consumed: 0 0 0
Produced: 0 0 0
Produced: 1 1 1
Consumed: 0 1 1
Produced: 2 2 2
Consumed: 2 2 2
Consumed: 3 3 2
Produced: 3 3 3
Consumed: 3 3 3
Produced: 4 4 4
Produced: 5 5 5
```

```
Albertos-MacBook-Pro:ProducerConsumerFixed alberto$ java ProducerConsumer
Produced: 0 0 0
Consumed: 0 0 0
Produced: 1 1 1
Consumed: 1 1 1
Produced: 2 2 2
Consumed: 2 2 2
Produced: 3 3 3
Consumed: 3 3 3
Produced: 4 4 4
Consumed: 4 4 4
```

Much better!